

zk-pop-solana-lean4-v1: a zero-knowledge proof of possession of a Lean 4 proof

Generated from the Lean development in this repository

Release `zk-pop-solana-lean4-v1`
manifest root `0x5f21038acec49e38005dc301e313dddb`

Abstract

This is the release document of `zk-pop-solana-lean4-v1`. It answers a single question: *can you prove that you have this Lean 4 proof, without handing it over?* The answer has two halves, and both are machine-checked. First, *what* “this proof” is is pinned: the release manifest gives the SHA-256 digest of each of the 93 source files of the development, and the Lean kernel checks that folding those digests produces the manifest root printed on this title page. Second, possession of the withheld material is proved in zero knowledge: the prover publishes one non-interactive certificate — commitment 163, announcements (23, 136), challenges (87, 91), responses (120, 200) over $\mathbb{Z}_{257}$ — whose Fiat–Shamir hash is bound to the manifest root, and Lean checks that it verifies, that any rewinding pair of such certificates extracts a run certifying the public statement (hence the theorem of the underlying Lean proof), and that the whole interaction is *equal* — not merely indistinguishable — for either of two genuinely different certified runs. Every theorem cited here is `sorry`-free and passes an axiom audit that admits only `propext`, `Classical.choice` and `Quot.sound`. The two standing assumptions — collision resistance of SHA-256 and computational binding of the commitment — are stated explicitly, as is the fact that the concrete instantiation runs over the toy field $\mathbb{Z}_{257}$ for readability rather than at a security parameter.

1 What is being claimed

Publishing a formal proof normally means publishing the proof. For the development in this repository that would mean publishing, among other things, the certified *run* of the Colosseum vault program: every opcode and every amount. The claim of this release is the strictly weaker, and much more useful, statement:

We hold a Lean 4 development whose sources hash to the published manifest root, and we hold the withheld witness that its zero-knowledge argument speaks about; an accepted certificate entails the mathematical theorem, and discloses nothing else.

Concretely the release consists of the following four checked facts, conjoined in the single Lean theorem `Solana.ZkPoPSolanaV1.zk_pop_solana_lean4_v1`.

1. **Identity of the artifact.** The SHA-256 digests of the 93 pinned files fold to the published root (`releaseRoot_eq`); the paths are pairwise distinct (`manifest_paths_nodup`); the modules carrying the headline theorems are among them (`manifest_covers_headline_modules`).
2. **Completeness.** The published certificate verifies against the Fiat–Shamir hash bound to

that root (`releaseCert_verifies`), and its numbers are exactly the ones printed in §5 (`releaseCert_numbers`, `releaseCert_public_data`, `releaseCert_equations`).

3. **Knowledge soundness.** Two certificates on the same commitment and announcements that verify under different hash values — the formal shadow of rewinding a reprogrammed random oracle — yield a run that really certifies the public final account, together with an opening of the commitment to it, and hence the claim of the original proof: the public final account decodes to a vault whose balance and lifetime totals are consistent (`releaseCert_sound`).
4. **Zero knowledge.** For every challenge, the joint law of the commitment and of the whole interaction is the same whichever certified run the prover holds (`releaseCert_zero_knowledge`); it is in fact equal to the output of a simulator that knows no secret at all (`releaseCert_simulatable`). The two runs in question differ in length, in their deposits and in their byte encodings, and both certify (`releaseCert_hides_the_run`).

2 The pinned artifact

The release manifest is generated by `tools/make_release_manifest.py` and shipped in two formats: `zkpop/artifact/zk-pop-solana-lean4-v1.manifest.sha256`, which `sha256sum -c` accepts directly, and `RequestProject/ReleaseManifestV1.lean`, which carries the same data as Lean terms (each digest as its hex string *and* as the natural number it denotes). Two files are excluded, because they are the release statement and would otherwise have to hash themselves: `RequestProject/ReleaseManifestV1.lean` and `RequestProject/ZkPoPSolanaV1.lean`.

The manifest root is the Horner fold of the digests, in manifest order, modulo the Mersenne prime $2^{127} - 1$:

$$\text{root} = \left(\left(\cdots (0 \cdot \mu + d_1) \cdot \mu + d_2 \cdots \right) \right) \bmod (2^{127} - 1), \quad \mu = 1000003,$$

where d_i is the i -th SHA-256 digest read as a natural number. Its value is

$$126448077240605503811882941449248169435 = 0x5f21038acec49e38005dc301e313dddb.$$

Theorem 1 (`releaseRoot_eq`). *`releaseRoot = publishedRoot`: folding the 93 digests of the shipped manifest produces the number above. Checked by the Lean kernel.*

The complete manifest follows; the full 64-hex-digit digests are in the shipped `.sha256` file.

file	SHA-256 (first 16 hex digits)
<code>RequestProject/ColosseumCertified.lean</code>	823eddc069b9a4a5
<code>RequestProject/Justifiable.lean</code>	935f07bd091c57f6
<code>RequestProject/Main.lean</code>	929b0bddef0b781f
<code>RequestProject/MonsterPepe.lean</code>	3ce0aa25c52ce761
<code>RequestProject/MoonshineMultiplicities.lean</code>	a67232ed221381a8
<code>RequestProject/Murmur3.lean</code>	cc943b9c66c2c3f5
<code>RequestProject/PaperManifest.lean</code>	c6dead45eff6b1ca
<code>RequestProject/ProofOfProof.lean</code>	be0c5c5ad3f35dd3
<code>RequestProject/PublicContracts.lean</code>	0ae096fcae9472be
<code>RequestProject/SBFMachine.lean</code>	64d3b0160d73127f
<code>RequestProject/SelfCertifying.lean</code>	749c6e9615c48559
<code>RequestProject/SelfCertifyingSBF.lean</code>	32965110fb848eec
<code>RequestProject/Solana.lean</code>	e513e2eb8bc78257

file	SHA-256 (first 16 hex digits)
RequestProject/SolanaPDA.lean	e0c3144940edea5a
RequestProject/SolanaToken.lean	48fca6ae1d989e25
RequestProject/SolanaTokenDelegate.lean	2a23e21ed617e7b5
RequestProject/SolanaVault.lean	e44010913943f121
RequestProject/Vault.lean	8211a814ed1b8e7e
RequestProject/ZkPoPofOurProof.lean	b598e3200784c992
RequestProject/ZkPoPRecursive.lean	c210bb62f1f28c71
RequestProject/ZkTokenSkill.lean	4b12ae071fea8e77
Solanalib.lean	d32184e09fe01546
Solanalib/ChainAbstraction/Ledger.lean	b069d563dcb78898
Solanalib/ChainAbstraction/ProofCertification.lean	8b100fa7d40472c0
Solanalib/ChainAbstraction/ProofValidator.lean	4d6cb7dd0075c7e2
Solanalib/Primitives/Pda.lean	08e3d69918af5eaf
Solanalib/Primitives/Pubkey.lean	7f17865aad5c5646
Solanalib/VM/SBF.lean	153e5117fa9c975e
SolanalibTest.lean	f93bb8b2b427c7c5
SolanalibTest/Primitives/PdaTest.lean	d9b0f655801b1c63
ZkPoP/Arch/Defect.lean	5dae9f075890b838
ZkPoP/Arch/Instance.lean	aca82d29e7aa2dd5
ZkPoP/Arch/Ladder.lean	7b663120bd9e919a
ZkPoP/Arch/Lattice.lean	b4084e82219517ce
ZkPoP/Arch/Packaging.lean	f05afbfc2f230a22
ZkPoP/Arch/Tower.lean	397f4f6c755bcc83
ZkPoP/Arith.lean	e1e3dabf49da818d
ZkPoP/Bridge.lean	47759fd2ca2bf886
ZkPoP/Cascade/Arith.lean	ddb6532b57825554
ZkPoP/Cascade/Distill.lean	44896dc64b88c290
ZkPoP/Cascade/Group.lean	2fd325fc91b07f22
ZkPoP/Cascade/Leak.lean	36e76cd20fc0a2fb
ZkPoP/Cascade/Monster.lean	e35da2e7b3c022ad
ZkPoP/Cascade/Report.lean	7c73eb138b534722
ZkPoP/Cascade/Zk.lean	978f710fcd5abb2a
ZkPoP/Catalog.lean	b036f8177f1328fe
ZkPoP/Circuit.lean	8c9d2fb817a9383d
ZkPoP/Core.lean	eb78609ddc480283
ZkPoP/CoreArith.lean	d9391819620d5092
ZkPoP/CoreReduction.lean	ea294bdcc1ce3a93
ZkPoP/CoreSemantics.lean	e8dd278e268792d8
ZkPoP/Decoder/Cli.lean	8480a6acee6730e3
ZkPoP/Decoder/Core.lean	74e820fdc9a17929
ZkPoP/Decoder/Pipeline.lean	0241b854d13a945c
ZkPoP/Decoder/Shards.lean	022d663c91d27d8a
ZkPoP/Lang.lean	83bcfa06d299b53f
ZkPoP/Main.lean	09450aef6a41cbbe
ZkPoP/Merged.lean	a7f8c27f4ffafabb
ZkPoP/Merged/Certificate.lean	627c8400c4385d7d
ZkPoP/Merged/Circuit.lean	20b1de41f5029cad
ZkPoP/Merged/CommitAndProve.lean	e2f434182f8c04b8
ZkPoP/Merged/Commitment.lean	a8635ecbd5111b2c
ZkPoP/Merged/DeclCircuit.lean	dfba6b4da8f1dc84
ZkPoP/Merged/Declaration.lean	99ecc9abb0982f83
ZkPoP/Merged/Demo.lean	30fe8a18178b41c8

file	SHA-256 (first 16 hex digits)
ZkPoP/Merged/Final.lean	2b04b08cd1e50ebb
ZkPoP/Merged/Language.lean	72d3123ce972a5b9
ZkPoP/Merged/OrProtocol.lean	e103fcae51026615
ZkPoP/Merged/Recursion.lean	f4773d936e11bc92
ZkPoP/Merged/Semantics.lean	88134f6a0d435105
ZkPoP/Merged/Shard.lean	fb3d42f0e31db2de
ZkPoP/Merged/Sigma.lean	93249b62cd1f39d9
ZkPoP/Pedersen.lean	a2cff2847b0fcc22
ZkPoP/Recursion.lean	a5a6157628107152
ZkPoP/Refined.lean	b5e45fca79146665
ZkPoP/Refined/GroupStructure.lean	0938e05c8ef45dd3
ZkPoP/Refined/Morphism.lean	770681f393f27805
ZkPoP/Refined/SemanticZK.lean	dd8bde2d4e305b85
ZkPoP/Report.lean	5ceb1fac3b81d78f
ZkPoP/SHHS/Audit.lean	ae77913cf13ac296
ZkPoP/SHHS/CatalogData.lean	8c70a0641db18a52
ZkPoP/SHHS/Coverage.lean	fe8f334a1ccd618a
ZkPoP/SHHS/Encoding.lean	4953afcae730678d
ZkPoP/SHHS/Meta.lean	bee7513b4cb8c38a
ZkPoP/SHHS/View.lean	46c00ad1adfaf26e
ZkPoP/Schnorr.lean	e44b9f0f973ff080
ZkPoP/SelfHost.lean	2b10e693aab1e6e9
ZkPoP/Shards.lean	725162889415d280
ZkPoP/Skill.lean	6b707d1b2c651842
ZkPoP/Universal.lean	59bed473ca93e675
lake-manifest.json	84965c68cec2c5a1
lakefile.lean	a6a385dd2b51f92b
lean-toolchain	db7bb24b756d745b

3 The statement being proved in zero knowledge

The semantic content comes from `RequestProject/ColosseumCertified.lean`: the Colosseum vault, as a self-certifying Solana program, is run by a sequence of transactions from the freshly initialised account `zeroAccount`; the run is *certified* when every transaction succeeds and it ends in a designated account. The public statement of the release is the final account

$$\text{publicFinalAccount} = \text{encodeVault} \langle 70, 100, 30 \rangle$$

— 70 held, 100 deposited in total, 30 withdrawn — which is the state the chain sees anyway. The withheld witness is a run reaching it. Two such runs form the public catalogue:

$$\text{runA} = [(0, 100), (1, 30)], \quad \text{runB} = [(0, 60), (0, 40), (1, 30)],$$

both certified (`runA_certifies`, `runB_certifies`), of different lengths and with different byte encodings.

The bridge from the witness to the mathematics is `certifies_vault_ok`: a certified run entails that `publicFinalAccount` decodes to a vault `v` with `v.ok = true`. That is the theorem the verifier walks away with.

4 The argument

The argument layer is the commit-and-prove construction of `RequestProject/ZkPoP0fOurProof.lean`, over the modules of `ZkPoP.Merged`. Fix a field K , a module N of commitments, a module R of blinding randomness, and an arithmetic encoding enc of secrets as D field cells. The prover publishes

$$C = \varphi(\text{enc } s) + \psi(\rho) \in N,$$

and runs the Cramer–Damgård–Schoenmakers OR-composition over the branch statements

$$y_j = C - \varphi(\text{enc}(\text{sec}_j)), \quad j < n,$$

branch j asserting “ C opens to the encoding of the j -th accepted secret”. A transcript is $(\text{ann}, \text{chal}, \text{resp})$ and the verifier accepts e iff

$$\sum_j \text{chal}_j = e \quad \text{and} \quad \psi(\text{resp}_j) = \text{ann}_j + \text{chal}_j \cdot y_j \quad \text{for all } j.$$

Completeness, special soundness (an explicit extractor) and perfect honest-verifier zero knowledge for a surjective ψ are theorems of that development, as is the Fiat–Shamir compilation into a single published certificate $\text{FSCert} = (C, \text{transcript})$ verified against a recomputed challenge $H(C, \text{ann})$.

5 The published certificate

The release instantiates the above at $K = N = R = \mathbb{Z}_{257}$ (byte values are distinct modulo 257, so the encoding of a run is faithful), $D = 32$ cells, $\varphi(v) = \sum_{k < 32} (k+1) v_k$, and $\psi = \text{id}$, which is surjective and therefore makes the commitment perfectly hiding. The Fiat–Shamir hash is bound to the manifest root:

$$H_{v1}(C, \text{ann}) = \text{root}_{\mathbb{F}} + 3C + 5 \text{ann}_0 + 7 \text{ann}_1, \quad \text{root}_{\mathbb{F}} = \text{root} \bmod 257 = 164.$$

The prover holds branch 0 (`runA`), with blinding $\rho = 137$, simulated challenge 91 on the other branch and own randomness $(23, 200)$. The published certificate is then:

commitment C	163
announcements $(\text{ann}_0, \text{ann}_1)$	(23, 136)
branch challenges $(\text{chal}_0, \text{chal}_1)$	(87, 91)
responses $(\text{resp}_0, \text{resp}_1)$	(120, 200)
branch statements (y_0, y_1)	(137, 12)
recomputed challenge $H_{v1}(C, \text{ann})$	178

Every entry is checked by the kernel (`releaseCert_numbers`, `releaseCert_public_data`), and so are the verifier’s equations (`releaseCert_equations`), which the reader can also redo by hand in $\mathbb{Z}_{257}$:

$$87 + 91 = 178, \quad 120 = 23 + 87 \cdot 137, \quad 200 = 136 + 91 \cdot 12.$$

Theorem 2 (`releaseCert_verifies`). *FSVerify holds of the published certificate: it verifies against H_{v1} .*

Theorem 3 (`releaseCert_sound`). *Let c, c' be certificates with the same commitment and the same announcements, verifying under hashes H, H' with $H(C, \text{ann}) \neq H'(C, \text{ann})$. Then there exist a run r certifying `publicFinalAccount` and a blinding ρ with $\text{com}(\text{enc } r, \rho) = C$; and `publicFinalAccount` decodes to a vault v with $v.\text{ok} = \text{true}$.*

Theorem 4 (`releaseCert_zero_knowledge`, `releaseCert_simulatable`). *For every challenge e , the joint law of the commitment and the interaction is the same for branch 0 and branch 1, and equals the output of the witness-free simulator.*

Theorem 5 (`releaseHash_binds_root`). *The Fiat–Shamir hash is injective in the manifest root: developments with different roots (modulo the characteristic) get different hashes, so a certificate cannot be replayed for a different artifact.*

6 What the release does *not* claim

Assumption 1 (Collision resistance). *The manifest root identifies the sources only under collision resistance of SHA-256. Nothing in Lean proves that; the Lean side proves that the shipped digests fold to the published root, and `sha256sum -c` connects the digests to actual bytes.*

Assumption 2 (Binding). *The commitment used here is perfectly hiding. By `Merged.perfect_hiding_not_binding` it therefore cannot also be unconditionally binding; binding is a computational assumption, which for Pedersen commitments reduces to discrete log (`Merged.pedersen_binding_extract_dlog`).*

Remark 1 (Toy parameters). $\mathbb{Z}_{257}$ has a 257-element challenge space. The concrete certificate is an *exhibition* at numbers a reader can recheck by hand, not a parameter choice with a security level. All completeness, soundness and zero-knowledge theorems are proved for an arbitrary field, an arbitrary commitment module and an arbitrary hash; only the exhibited numbers are specific.

Remark 2 (Recursion transports soundness). The repository also contains a recursive tower (`RequestProject/ZkPoPRecursive.lean`): proofs of knowledge of proofs, to any depth, with completeness at every depth (`recursive_zkpop_completeness`), descent (`accK_descend`) and soundness back to the base relation (`recursive_zkpop_sound`). As recorded there and in `paper/zkpop.tex`, recursion *transports* soundness rather than creating it: the semantic content always comes from the base theorem about certified runs.

7 How to verify this release

1. `sha256sum -c zkpop/artifact/zk-pop-solana-lean4-v1.manifest.sha256` — the shipped sources are the pinned ones.
2. `python3 tools/make_release_manifest.py` — regenerates the manifest and the Lean data file; they must be unchanged.
3. `lake build RequestProject.ZkPoPSolanaV1` — re-elaborates the release module. This kernel-checks the manifest root, the certificate numbers, the verifier’s equations, completeness, knowledge soundness and zero knowledge, and runs the axiom audit of `RequestProject/PaperManifest.lean`, which fails the build if any release theorem depends on anything beyond `propext`, `Classical.choice` and `Quot.sound` — in particular if any depends on `sorry` or on `native_decide`.

All three steps are wrapped in `tools/verify-release.sh`.

8 Index of the release theorems

Lean name (namespace <code>Solana.ZkPoPSolanaV1</code>)	content
<code>releaseRoot_eq</code>	manifest folds to the published root
<code>manifest_fileCount</code>	93 files pinned
<code>manifest_paths_nodup</code>	no file pinned twice
<code>manifest_covers_headline_modules</code>	headline modules are pinned
<code>releaseHash_binds_root</code>	the hash is injective in the root
<code>releaseCert_numbers</code>	the published numbers
<code>releaseCert_public_data</code>	branch statements and challenge
<code>releaseCert_equations</code>	the verifier's equations hold
<code>releaseCert_verifies</code>	completeness
<code>releaseCert_sound</code>	knowledge soundness, and the vault theorem
<code>releaseCert_zero_knowledge</code>	perfect zero knowledge
<code>releaseCert_simulatable</code>	the view is simulatable
<code>releaseCert_hides_the_run</code>	the two witnesses are different, both certified
<code>zk_pop_solana_lean4_v1</code>	the release, in one statement